

Prob_ODE_test_higher_dim

January 31, 2022

```
[1]: # Make inline plots vector graphics instead of raster graphics
      %matplotlib inline
      from IPython.display import set_matplotlib_formats

      set_matplotlib_formats("pdf", "svg")

      # Plotting
      import matplotlib.pyplot as plt
```

```
[2]: import numpy as np
      from probnum.diffeq import probsolve_ivp
```

```
[ ]:
```

1 Three-dimensional linear system

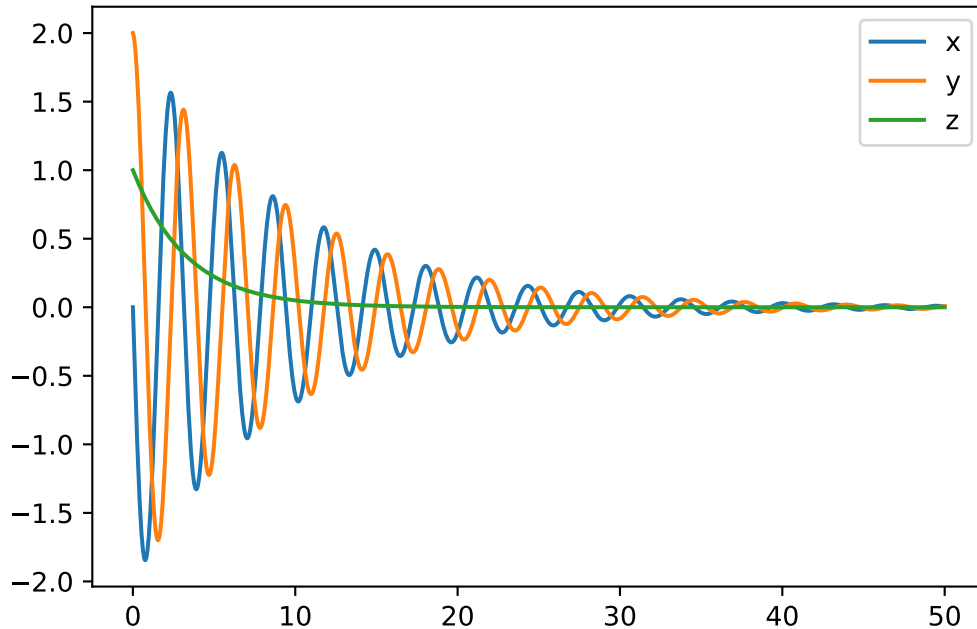
```
[3]: def f(t, y):
      y1, y2, y3 = y
      return np.array([-0.1*y1-2*y2, 2*y1-0.1*y2, -0.3*y3])

      def df(t, y):
          y1, y2, y3 = y
          return np.array([[-0.1, 2, 0],[2, -0.1, 0],[0,0,-0.3]])
```

```
[4]: t0 = 0.0
      tmax = 50.0
      y0 = np.array([0,2,1])
```

```
[5]: # IBM(1) with EKO, solving an IVP with adaptive steps works.
      sol = probsolve_ivp(f, t0, tmax, y0, df=df, step=0.1, adaptive=False,
      ↪diffusion_model="dynamic")
      dt=0.01
      evalgrid = np.arange(0,50, dt)
      means = sol(evalgrid).mean
```

```
[6]: plt.plot(evalgrid, means[:, 0], label="x")
plt.plot(evalgrid, means[:, 1], label="y")
plt.plot(evalgrid, means[:, 2], label="z")
plt.legend(loc="upper right")
plt.show()
```



```
[7]: from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure(figsize=(4, 4))
ax = fig.add_subplot(1, 1, 1, projection="3d")
ax.plot(
    means[:, 0],
    means[:, 1],
    means[:, 2]
)
ax.set(title='3D plot', xlabel="$x$", ylabel="$y$", zlabel="$z$")
ax.legend()
fig.show()
```

No handles with labels found to put in legend.

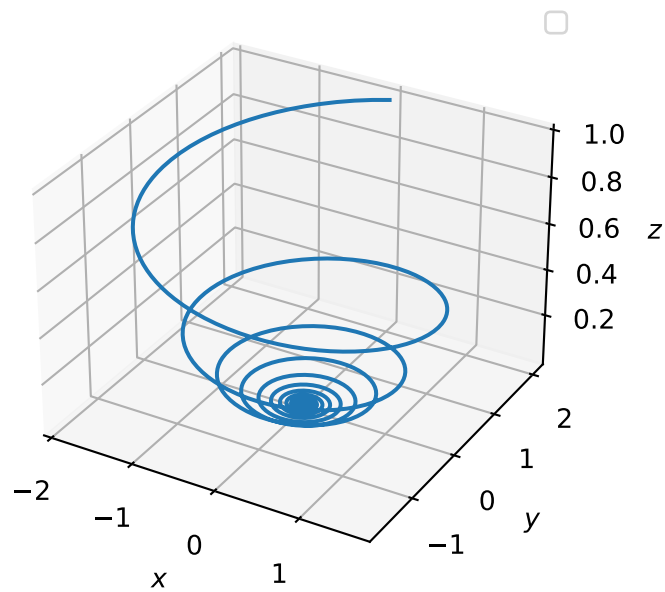
/home/kkumar/anaconda3/lib/python3.7/site-packages/ipykernel_launcher.py:11:

UserWarning: Matplotlib is currently using

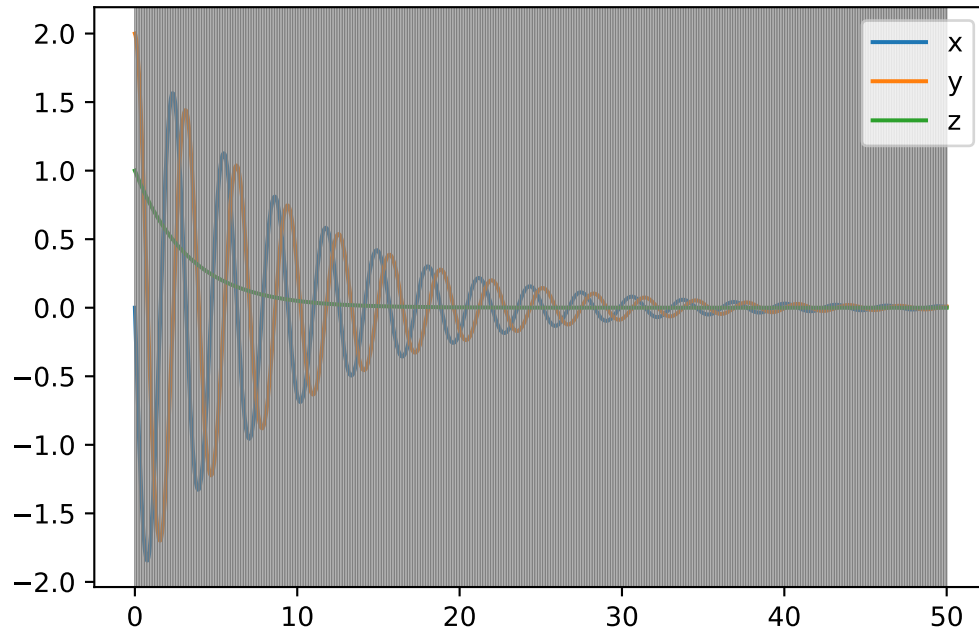
module://ipykernel.pylab.backend_inline, which is a non-GUI backend, so cannot show the figure.

This is added back by InteractiveShellApp.init_path()

3D plot



```
[8]: # Let's visualise the individual steps.
plt.plot(evalgrid, means[:, 0], label="x")
plt.plot(evalgrid, means[:, 1], label="y")
plt.plot(evalgrid, means[:, 2], label="z")
for t in sol.locations:
    plt.axvline(t, linewidth=0.5, color="gray")
plt.legend(loc="upper right")
plt.show()
```

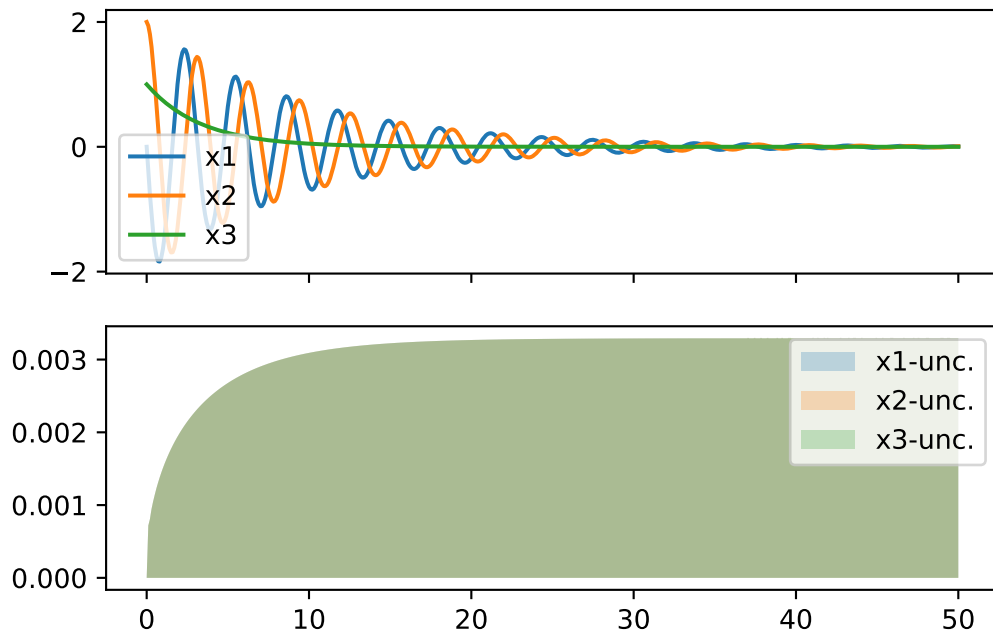


```
[9]: sol = probsolve_ivp(
    f, t0, tmax, y0, df=df, step=0.1, adaptive=False, diffusion_model="dynamic"
)

means, stds = sol.states.mean, sol.states.std

fig, (ax1, ax2) = plt.subplots(nrows=2, ncols=1, sharex=True)
ax1.plot(sol.locations, means[:, 0], label="x1")
ax1.plot(sol.locations, means[:, 1], label="x2")
ax1.plot(sol.locations, means[:, 2], label="x3")
ax1.legend()
ax2.fill_between(sol.locations, stds[:, 0], alpha=0.25, label="x1-unc.")
ax2.fill_between(sol.locations, stds[:, 1], alpha=0.25, label="x2-unc.")
ax2.fill_between(sol.locations, stds[:, 2], alpha=0.25, label="x3-unc.")
ax2.legend()
fig.suptitle("EKO Solution")
plt.show()
```

EK0 Solution



```
[10]: import numpy as np

from probnum.diffeq import probsolve_ivp

rng = np.random.default_rng(seed=123)

num_samples = 150
locations = np.arange(0.0, 22.1, 0.05)
locations = sol.locations
samples = sol.sample(rng=rng, t=locations, size=num_samples)

solution = sol(locations)
means = solution.mean
stds = solution.std

fig, (ax1) = plt.subplots(nrows=1, ncols=1, sharex=True)
for sample in samples[:5]:
    ax1.plot(locations, sample[:, 0], ":", color="C0", linewidth=0.5, alpha=1)
    ax1.plot(locations, sample[:, 1], ":", color="C1", linewidth=0.5, alpha=1)
    ax1.plot(locations, sample[:, 2], ":", color="C2", linewidth=0.5, alpha=1)
ax1.plot(locations, means[:, 0], label="x1", color="k")
ax1.plot(locations, means[:, 1], label="x2", color="k")
```

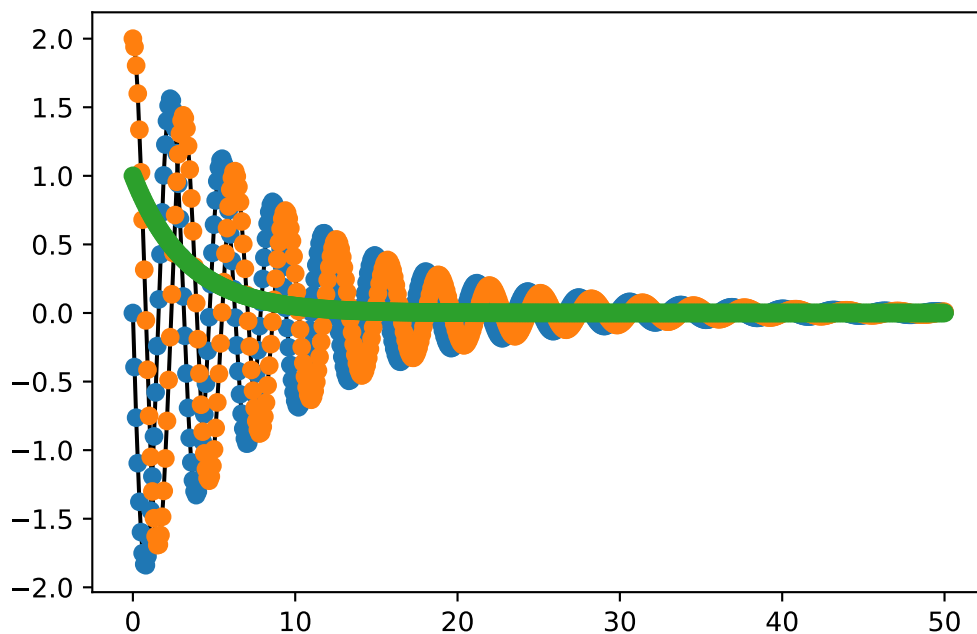
```

ax1.plot(locations, means[:, 2], label="x2", color="k")
ax1.fill_between(
    locations,
    means[:, 0] - 3 * stds[:, 0],
    means[:, 0] + 3 * stds[:, 0],
    color="C0",
    alpha=0.3,
)
ax1.fill_between(
    locations,
    means[:, 1] - 3 * stds[:, 1],
    means[:, 1] + 3 * stds[:, 1],
    color="C1",
    alpha=0.3,
)
ax1.fill_between(
    locations,
    means[:, 2] - 3 * stds[:, 2],
    means[:, 2] + 3 * stds[:, 2],
    color="C2",
    alpha=0.3,
)
ax1.plot(sol.locations, sol.states.mean, "o")
fig.suptitle(f"Samples From an EK0 Solution")

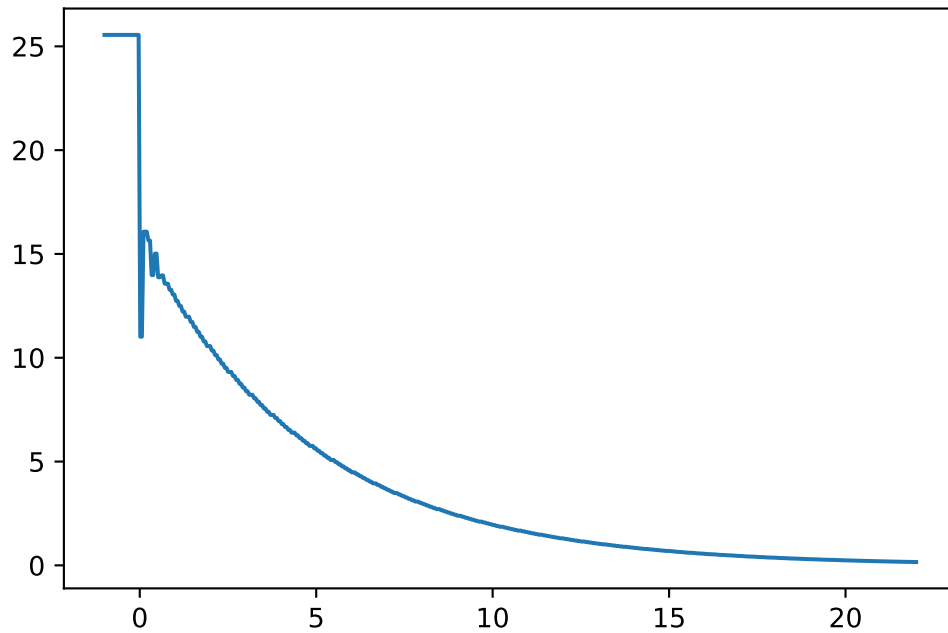
plt.show()

```

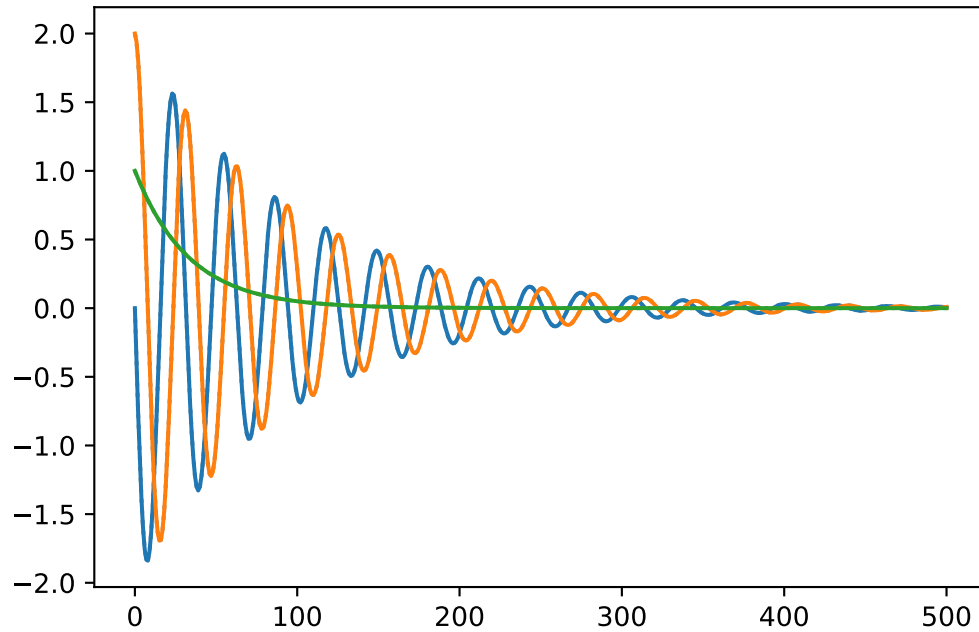
Samples From an EK0 Solution



```
[11]: x = np.linspace(-1, 22, 500)
plt.plot(x, sol.kalman_posterior.diffusion_model(x))
plt.show()
```



```
[12]: plt.plot(samples.mean(axis=0), color="gray", linestyle="dotted")
plt.plot(means)
plt.show()
```

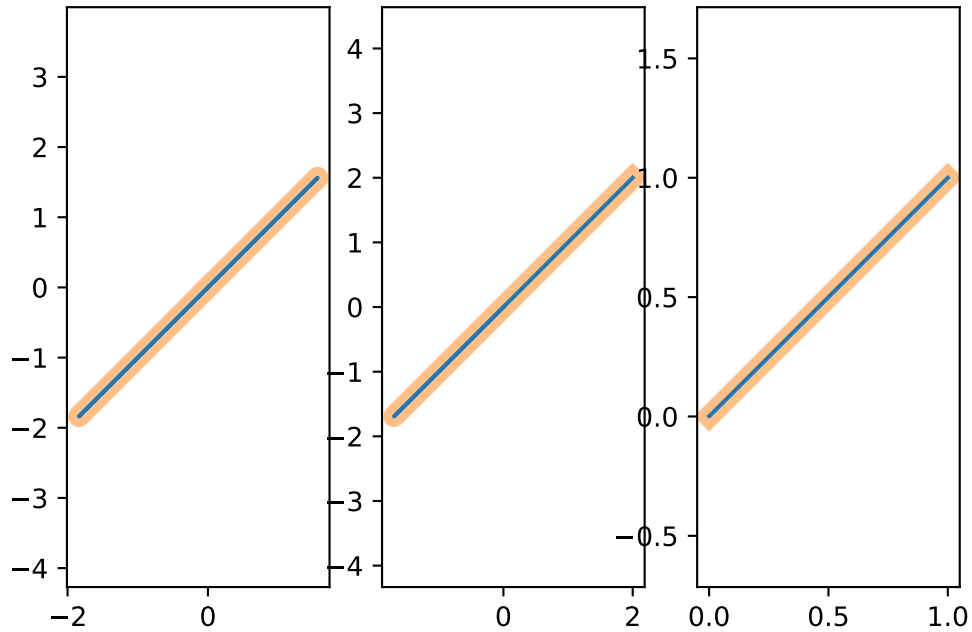


```
[13]: sample_mean = samples.mean(axis=0)
      sample_std = samples.std(axis=0)

      fig, (ax1, ax2, ax3) = plt.subplots(ncols=3)

      ax1.plot(means[:, 0], sample_mean[:, 0])
      ax2.plot(means[:, 1], sample_mean[:, 1])
      ax3.plot(means[:, 2], sample_mean[:, 2])
      ax1.plot(means[:, 0], means[:, 0], alpha=0.5, linewidth=8, zorder=0)
      ax2.plot(means[:, 1], means[:, 1], alpha=0.5, linewidth=8, zorder=0)
      ax3.plot(means[:, 2], means[:, 2], alpha=0.5, linewidth=8, zorder=0)

      ax1.axis("equal")
      ax2.axis("equal")
      ax3.axis("equal")
      plt.show()
```

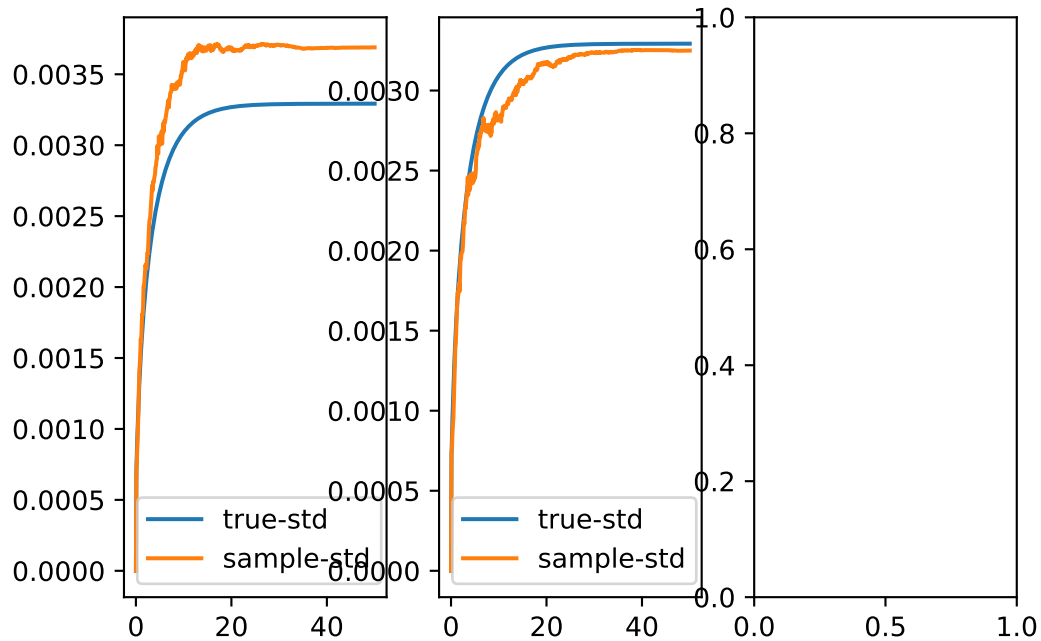


```
[14]: fig, (ax1, ax2, ax3) = plt.subplots(ncols=3)

ax1.plot(locations, stds[:, 0], label="true-std")
ax1.plot(locations, sample_std[:, 0], label="sample-std")
ax1.legend()

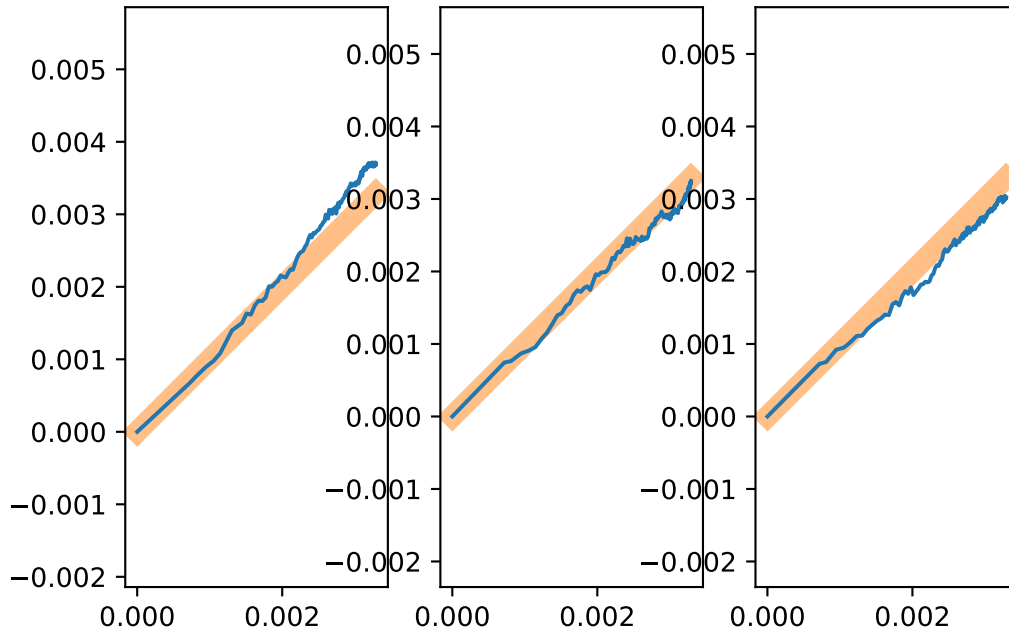
ax2.plot(locations, stds[:, 1], label="true-std")
ax2.plot(locations, sample_std[:, 1], label="sample-std")
ax2.legend()
plt.show()

ax3.plot(locations, stds[:, 2], label="true-std")
ax3.plot(locations, sample_std[:, 2], label="sample-std")
ax3.legend()
plt.show()
```



```
[15]: fig, (ax1, ax2, ax3) = plt.subplots(ncols=3)

ax1.plot(stds[:, 0], sample_std[:, 0])
ax2.plot(stds[:, 1], sample_std[:, 1])
ax3.plot(stds[:, 2], sample_std[:, 2])
ax1.plot(stds[:, 0], stds[:, 0], alpha=0.5, linewidth=8, zorder=0)
ax2.plot(stds[:, 1], stds[:, 1], alpha=0.5, linewidth=8, zorder=0)
ax3.plot(stds[:, 2], stds[:, 2], alpha=0.5, linewidth=8, zorder=0)
ax1.axis("equal")
ax2.axis("equal")
ax3.axis("equal")
plt.show()
```



[]:

2 Rossler Systems

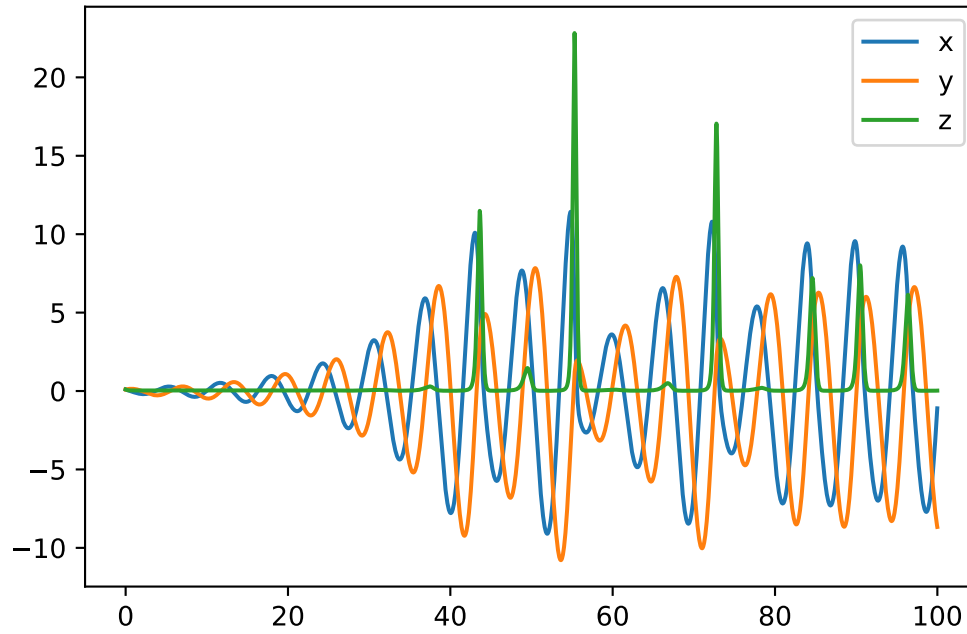
```
[16]: def f(t, y):
        y1, y2, y3 = y
        return np.array([-y2-y3, y1+0.2*y2, 0.2+y3*(y1-5.7)])

        def df(t, y):
            y1, y2, y3 = y
            return np.array([[0, -1, -1], [1, 0.2, 0], [y3, 0, y1-5.7]])
```

```
[17]: t0 = 0.0
        tmax = 100.0
        y0 = np.array([0.1, 0.1, 0.1])
```

```
[18]: # IBM(1) with EKO, solving an IVP with adaptive steps works.
        sol = probsolve_ivp(f, t0, tmax, y0, df=df, step=0.01, adaptive=False,
        ↪diffusion_model="dynamic")
        dt=0.01
        evalgrid = np.arange(0, 100, dt)
        means = sol(evalgrid).mean
```

```
[19]: plt.plot(evalgrid, means[:, 0], label="x")
plt.plot(evalgrid, means[:, 1], label="y")
plt.plot(evalgrid, means[:, 2], label="z")
plt.legend(loc="upper right")
plt.show()
```



```
[20]: from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure(figsize=(4, 4))
ax = fig.add_subplot(1, 1, 1, projection="3d")
ax.plot(
    means[:, 0],
    means[:, 1],
    means[:, 2]
)
ax.set(title='3D plot', xlabel="$x$", ylabel="$y$", zlabel="$z$")
ax.legend()
fig.show()
```

No handles with labels found to put in legend.

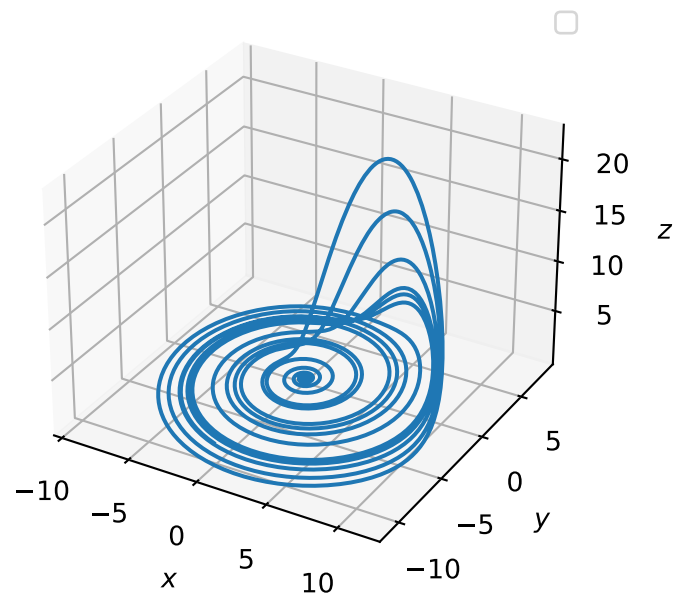
/home/kkumar/anaconda3/lib/python3.7/site-packages/ipykernel_launcher.py:11:

UserWarning: Matplotlib is currently using

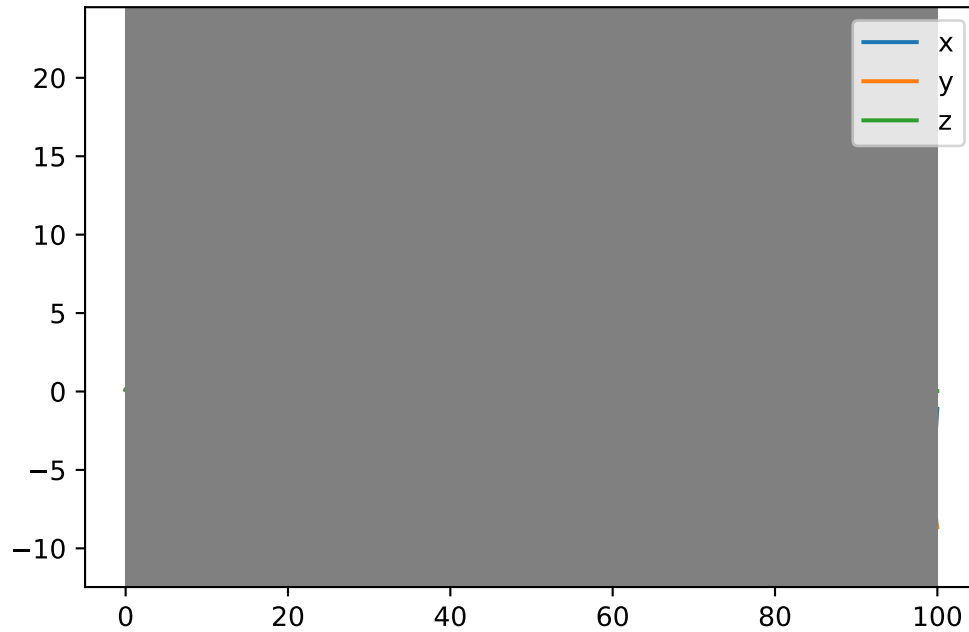
module://ipykernel.pylab.backend_inline, which is a non-GUI backend, so cannot show the figure.

This is added back by InteractiveShellApp.init_path()

3D plot



```
[21]: # Let's visualise the individual steps.
plt.plot(evalgrid, means[:, 0], label="x")
plt.plot(evalgrid, means[:, 1], label="y")
plt.plot(evalgrid, means[:, 2], label="z")
for t in sol.locations:
    plt.axvline(t, linewidth=0.5, color="gray")
plt.legend(loc="upper right")
plt.show()
```

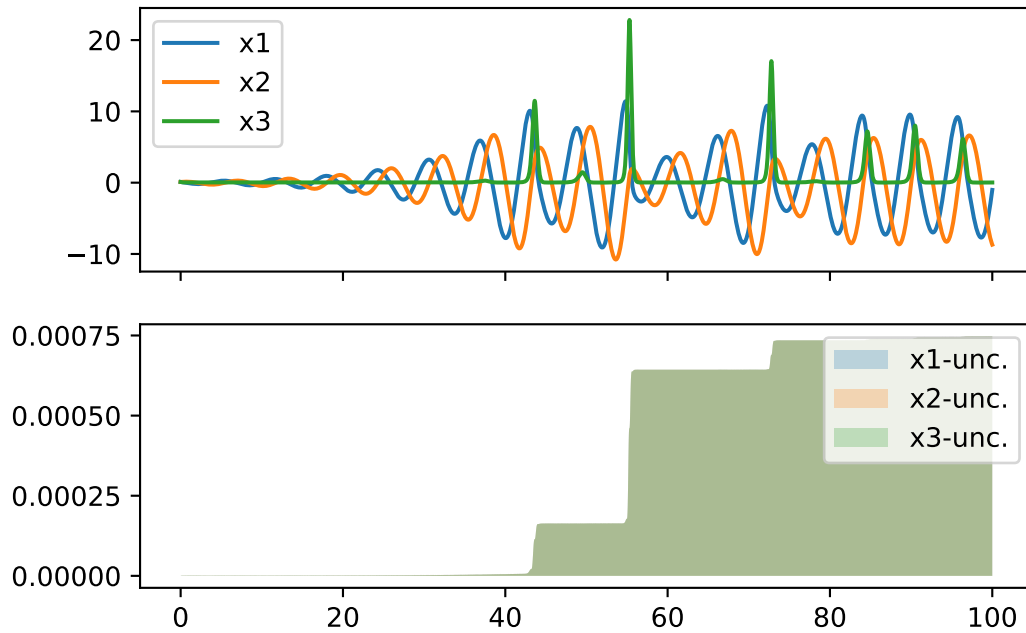


```
[22]: sol = probsolve_ivp(
        f, t0, tmax, y0, df=df, step=0.01, adaptive=False, diffusion_model="dynamic"
    )

means, stds = sol.states.mean, sol.states.std

fig, (ax1, ax2) = plt.subplots(nrows=2, ncols=1, sharex=True)
ax1.plot(sol.locations, means[:, 0], label="x1")
ax1.plot(sol.locations, means[:, 1], label="x2")
ax1.plot(sol.locations, means[:, 2], label="x3")
ax1.legend()
ax2.fill_between(sol.locations, stds[:, 0], alpha=0.25, label="x1-unc.")
ax2.fill_between(sol.locations, stds[:, 1], alpha=0.25, label="x2-unc.")
ax2.fill_between(sol.locations, stds[:, 2], alpha=0.25, label="x3-unc.")
ax2.legend()
fig.suptitle("EKO Solution")
plt.show()
```

EK0 Solution



```
[23]: import numpy as np

from probnum.diffeq import probsolve_ivp

rng = np.random.default_rng(seed=123)

num_samples = 150
locations = np.arange(0.0, 22.1, 0.05)
locations = sol.locations
samples = sol.sample(rng=rng, t=locations, size=num_samples)

solution = sol(locations)
means = solution.mean
stds = solution.std

fig, (ax1) = plt.subplots(nrows=1, ncols=1, sharex=True)
for sample in samples[::5]:
    ax1.plot(locations, sample[:, 0], ":", color="C0", linewidth=0.5, alpha=1)
    ax1.plot(locations, sample[:, 1], ":", color="C1", linewidth=0.5, alpha=1)
    ax1.plot(locations, sample[:, 2], ":", color="C2", linewidth=0.5, alpha=1)
ax1.plot(locations, means[:, 0], label="x1", color="k")
ax1.plot(locations, means[:, 1], label="x2", color="k")
```

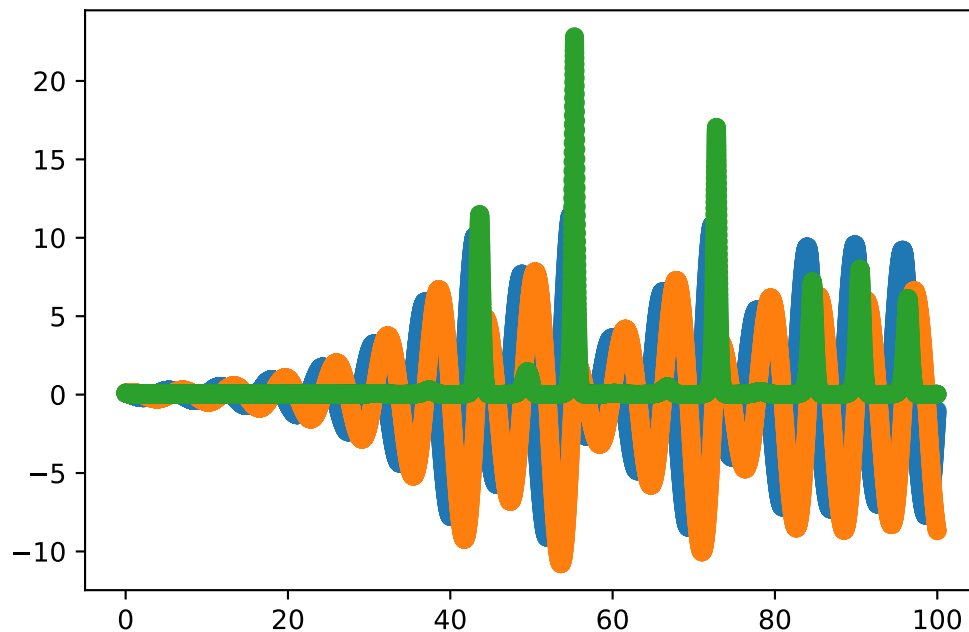
```

ax1.plot(locations, means[:, 2], label="x2", color="k")
ax1.fill_between(
    locations,
    means[:, 0] - 3 * stds[:, 0],
    means[:, 0] + 3 * stds[:, 0],
    color="C0",
    alpha=0.3,
)
ax1.fill_between(
    locations,
    means[:, 1] - 3 * stds[:, 1],
    means[:, 1] + 3 * stds[:, 1],
    color="C1",
    alpha=0.3,
)
ax1.fill_between(
    locations,
    means[:, 2] - 3 * stds[:, 2],
    means[:, 2] + 3 * stds[:, 2],
    color="C2",
    alpha=0.3,
)
ax1.plot(sol.locations, sol.states.mean, "o")
fig.suptitle(f"Samples From an EK0 Solution")

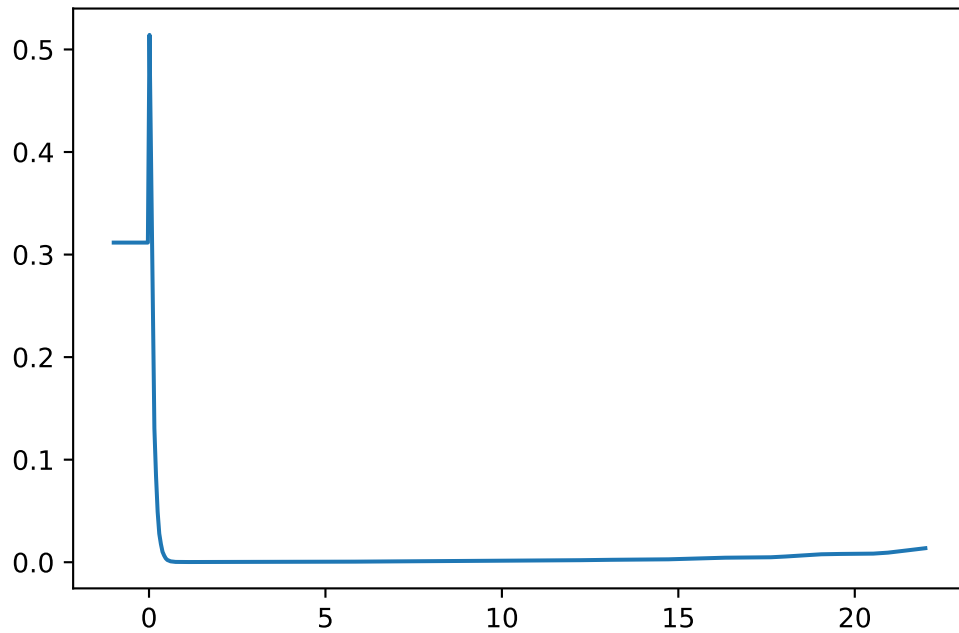
plt.show()

```

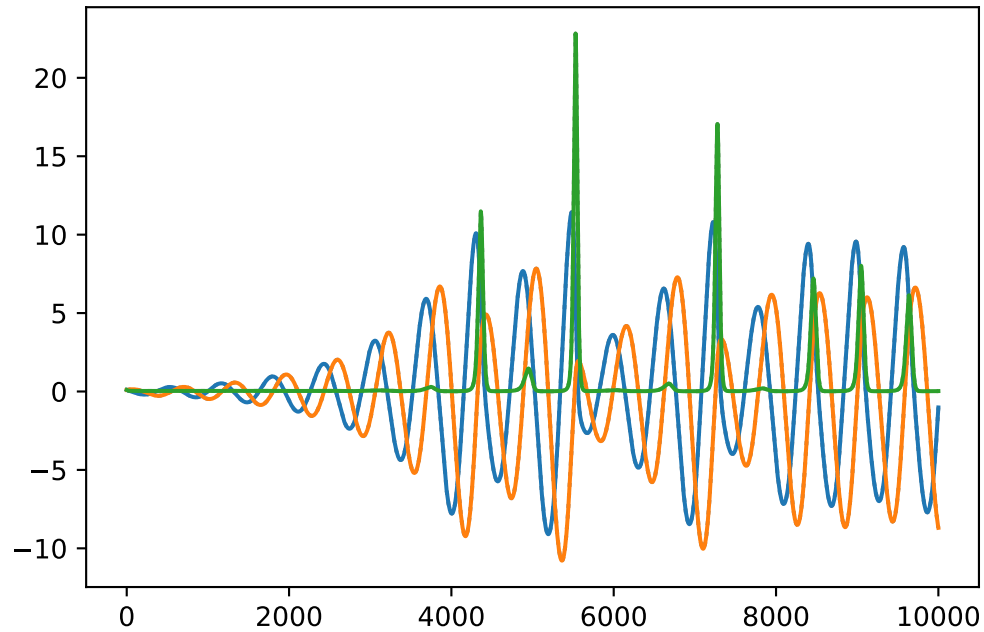
Samples From an EK0 Solution



```
[24]: x = np.linspace(-1, 22, 500)
plt.plot(x, sol.kalman_posterior.diffusion_model(x))
plt.show()
```



```
[25]: plt.plot(samples.mean(axis=0), color="gray", linestyle="dotted")
plt.plot(means)
plt.show()
```

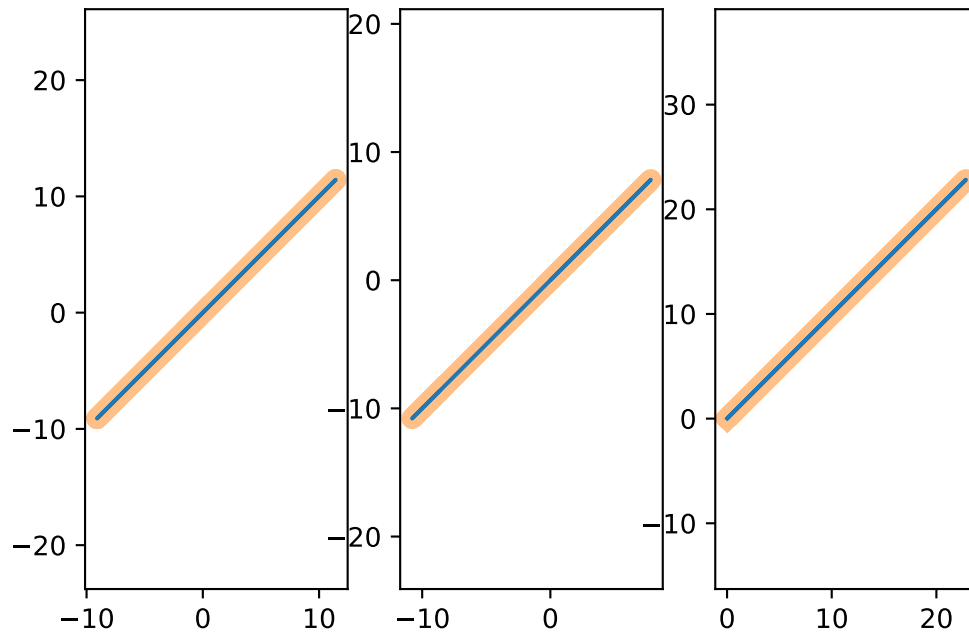


```
[26]: sample_mean = samples.mean(axis=0)
sample_std = samples.std(axis=0)

fig, (ax1, ax2, ax3) = plt.subplots(ncols=3)

ax1.plot(means[:, 0], sample_mean[:, 0])
ax2.plot(means[:, 1], sample_mean[:, 1])
ax3.plot(means[:, 2], sample_mean[:, 2])
ax1.plot(means[:, 0], means[:, 0], alpha=0.5, linewidth=8, zorder=0)
ax2.plot(means[:, 1], means[:, 1], alpha=0.5, linewidth=8, zorder=0)
ax3.plot(means[:, 2], means[:, 2], alpha=0.5, linewidth=8, zorder=0)

ax1.axis("equal")
ax2.axis("equal")
ax3.axis("equal")
plt.show()
```

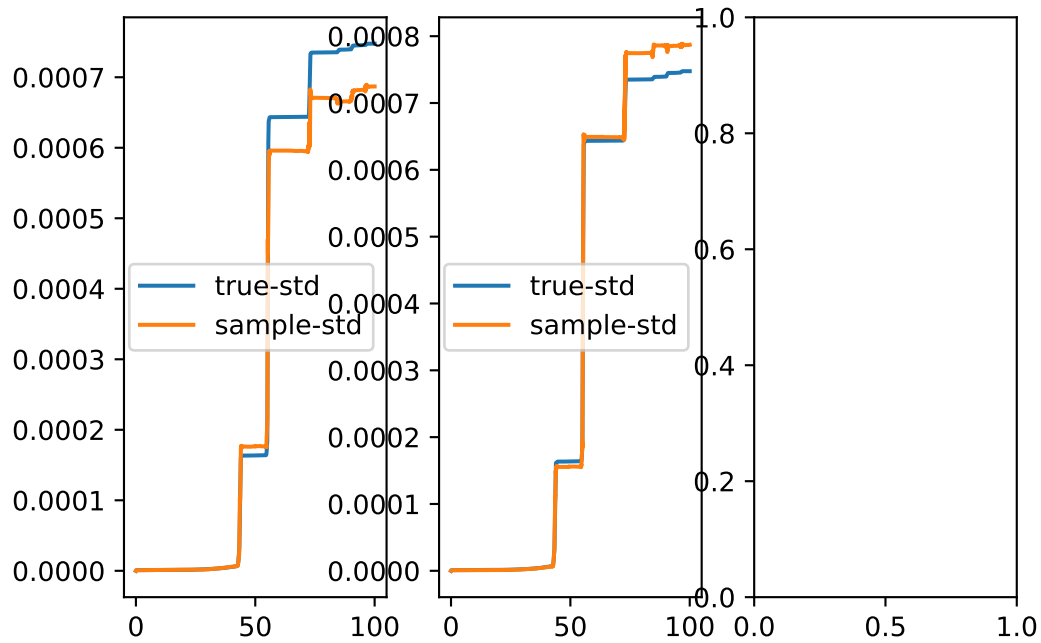


```
[27]: fig, (ax1, ax2, ax3) = plt.subplots(ncols=3)

ax1.plot(locations, stds[:, 0], label="true-std")
ax1.plot(locations, sample_std[:, 0], label="sample-std")
ax1.legend()

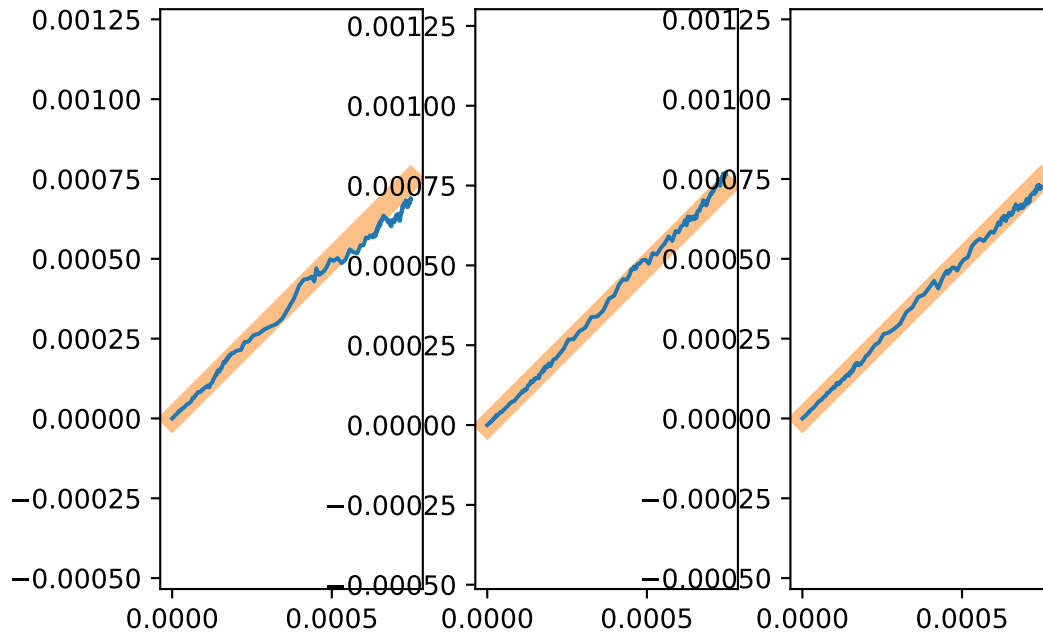
ax2.plot(locations, stds[:, 1], label="true-std")
ax2.plot(locations, sample_std[:, 1], label="sample-std")
ax2.legend()
plt.show()

ax3.plot(locations, stds[:, 2], label="true-std")
ax3.plot(locations, sample_std[:, 2], label="sample-std")
ax3.legend()
plt.show()
```



```
[28]: fig, (ax1, ax2, ax3) = plt.subplots(ncols=3)

ax1.plot(stds[:, 0], sample_std[:, 0])
ax2.plot(stds[:, 1], sample_std[:, 1])
ax3.plot(stds[:, 2], sample_std[:, 2])
ax1.plot(stds[:, 0], stds[:, 0], alpha=0.5, linewidth=8, zorder=0)
ax2.plot(stds[:, 1], stds[:, 1], alpha=0.5, linewidth=8, zorder=0)
ax3.plot(stds[:, 2], stds[:, 2], alpha=0.5, linewidth=8, zorder=0)
ax1.axis("equal")
ax2.axis("equal")
ax3.axis("equal")
plt.show()
```



[]:

3 Hopf Bifurcation

```
[29]: def f(t, y):
        y1, y2 = y
        return np.array([0,])

def df(t, y):
    y1, y2, y3 = y
    return np.array([[0,-1,-1],[1,0.2,0],[y3,0,y1-5.7]])
```

```
[30]: def f(t,y):
        y1,y2,y3=y
        mu =y1
        omega = 1
        A = 1

        f1 = 0
        f2 = mu*y2 + omega*y3 - A*y2*(y2**2+y3**2)
        f3 = -omega*y2 + mu*y3 - A*y3*(y2**2+y3**2)
```

```

    f = np.array([f1,f2,f3])
    return f

def df(t, y):
    y1,y2,y3=y
    mu =y1
    omega = 1
    A = 1
    return np.
    ↪array([0,0,0],[y2,mu-A*(y2**2+y3**2)-2*A*y2**2,omega-A*y2*2*y3],[y3,-omega-2*A*y2*y3,mu-A*

```

```

[31]: t0 = 0.0
      tmax = 75.0
      # initial condition
      y0 = np.array([-0.15,2,0])

```

```

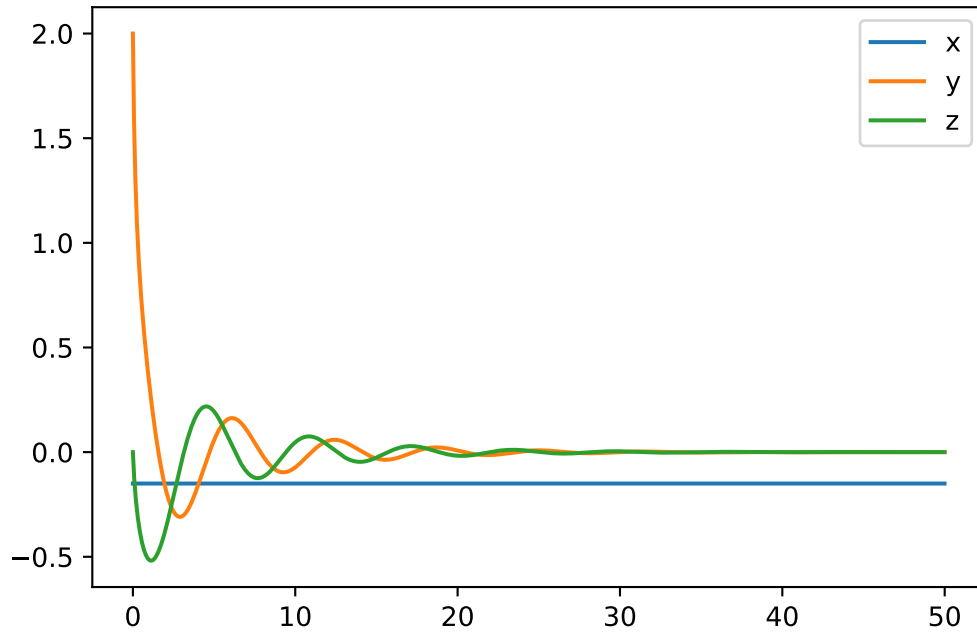
[32]: # IBM(1) with EKO, solving an IVP with adaptive steps works.
      sol = probsolve_ivp(f, t0, tmax, y0, df=df, step=0.01, adaptive=False,
      ↪diffusion_model="dynamic")
      dt=0.01
      evalgrid = np.arange(0,50, dt)
      means = sol(evalgrid).mean

```

```

[33]: plt.plot(evalgrid, means[:, 0], label="x")
      plt.plot(evalgrid, means[:, 1], label="y")
      plt.plot(evalgrid, means[:, 2], label="z")
      plt.legend(loc="upper right")
      plt.show()

```



```
[34]: from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure(figsize=(4, 4))
ax = fig.add_subplot(1, 1, 1, projection="3d")
ax.plot(
    means[:, 0],
    means[:, 1],
    means[:, 2]
)
ax.set(title='3D plot', xlabel="$x$", ylabel="$y$", zlabel="$z$")
ax.legend()
fig.show()
```

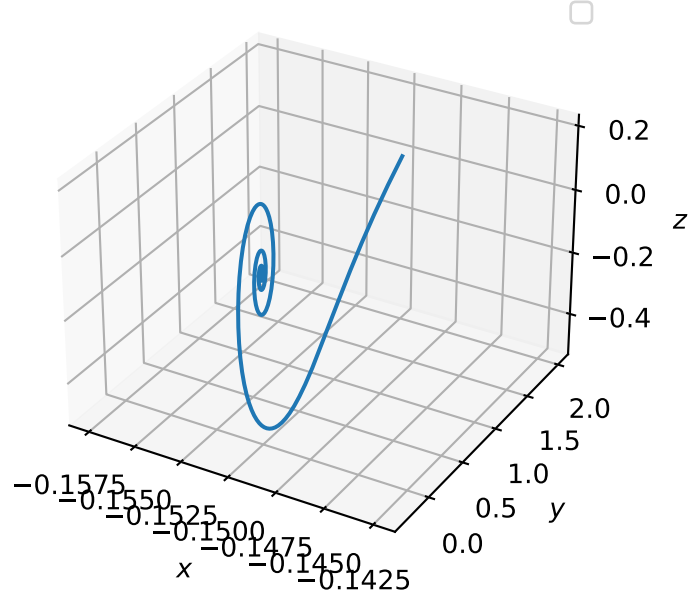
No handles with labels found to put in legend.

/home/kkumar/anaconda3/lib/python3.7/site-packages/ipykernel_launcher.py:11:

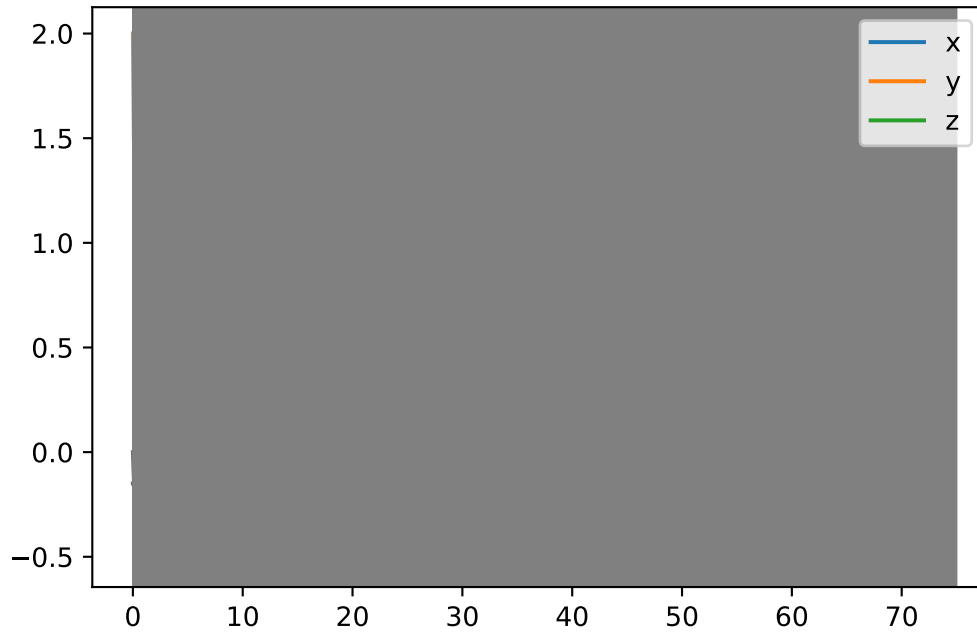
UserWarning: Matplotlib is currently using
module://ipykernel.pylab.backend_inline, which is a non-GUI backend, so cannot
show the figure.

This is added back by InteractiveShellApp.init_path()

3D plot



```
[35]: # Let's visualise the individual steps.
plt.plot(evalgrid, means[:, 0], label="x")
plt.plot(evalgrid, means[:, 1], label="y")
plt.plot(evalgrid, means[:, 2], label="z")
for t in sol.locations:
    plt.axvline(t, linewidth=0.5, color="gray")
plt.legend(loc="upper right")
plt.show()
```

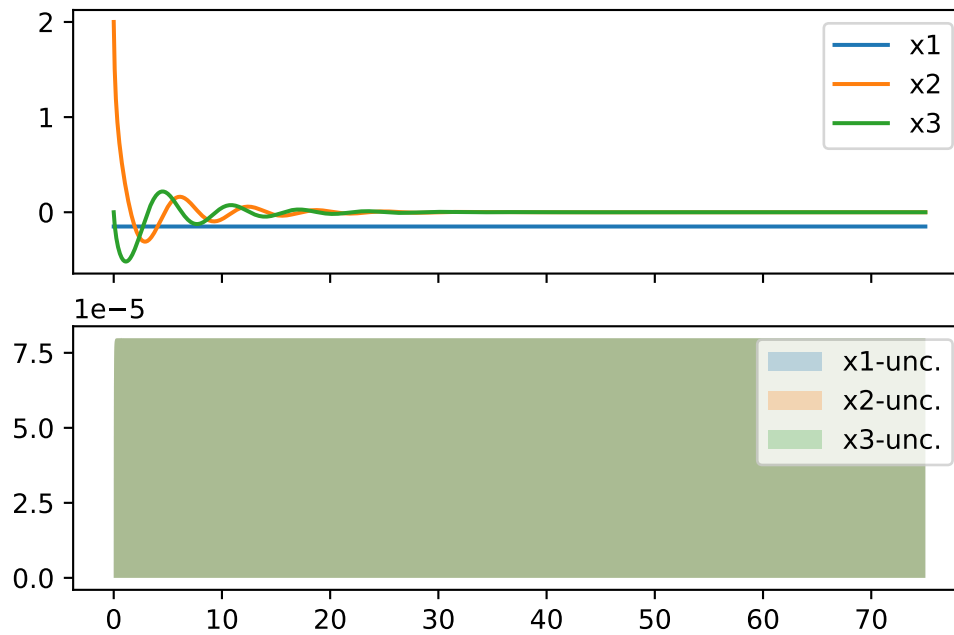


```
[36]: sol = probsolve_ivp(
    f, t0, tmax, y0, df=df, step=0.01, adaptive=False, diffusion_model="dynamic"
)

means, stds = sol.states.mean, sol.states.std

fig, (ax1, ax2) = plt.subplots(nrows=2, ncols=1, sharex=True)
ax1.plot(sol.locations, means[:, 0], label="x1")
ax1.plot(sol.locations, means[:, 1], label="x2")
ax1.plot(sol.locations, means[:, 2], label="x3")
ax1.legend()
ax2.fill_between(sol.locations, stds[:, 0], alpha=0.25, label="x1-unc.")
ax2.fill_between(sol.locations, stds[:, 1], alpha=0.25, label="x2-unc.")
ax2.fill_between(sol.locations, stds[:, 2], alpha=0.25, label="x3-unc.")
ax2.legend()
fig.suptitle("EKO Solution")
plt.show()
```

EK0 Solution



```
[37]: import numpy as np

from probnum.diffeq import probsolve_ivp

rng = np.random.default_rng(seed=123)

num_samples = 150
locations = np.arange(0.0, 22.1, 0.05)
locations = sol.locations
samples = sol.sample(rng=rng, t=locations, size=num_samples)

solution = sol(locations)
means = solution.mean
stds = solution.std

fig, (ax1) = plt.subplots(nrows=1, ncols=1, sharex=True)
for sample in samples[::5]:
    ax1.plot(locations, sample[:, 0], ":", color="C0", linewidth=0.5, alpha=1)
    ax1.plot(locations, sample[:, 1], ":", color="C1", linewidth=0.5, alpha=1)
    ax1.plot(locations, sample[:, 2], ":", color="C2", linewidth=0.5, alpha=1)
ax1.plot(locations, means[:, 0], label="x1", color="k")
ax1.plot(locations, means[:, 1], label="x2", color="k")
```

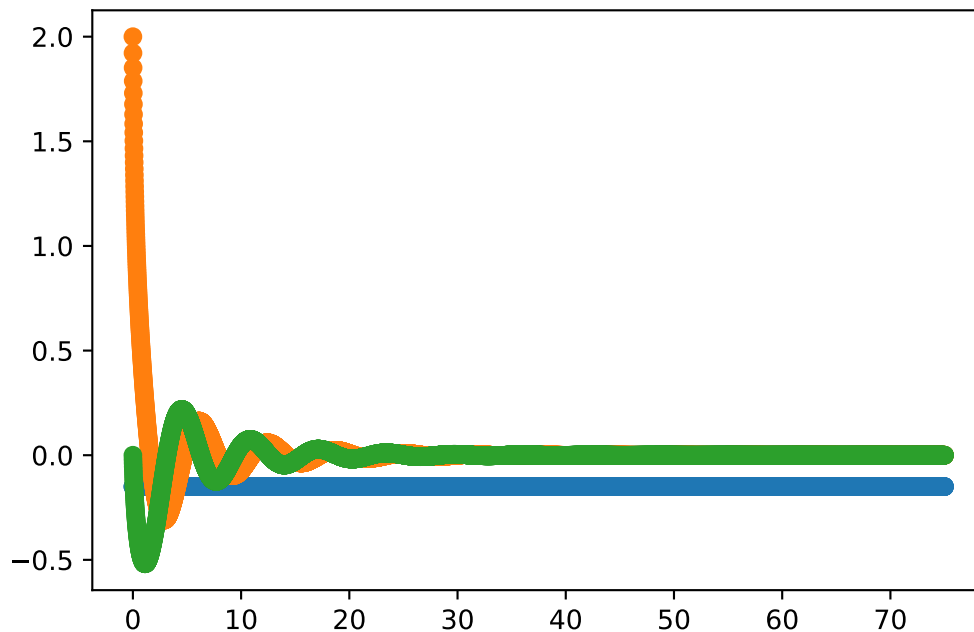
```

ax1.plot(locations, means[:, 2], label="x2", color="k")
ax1.fill_between(
    locations,
    means[:, 0] - 3 * stds[:, 0],
    means[:, 0] + 3 * stds[:, 0],
    color="C0",
    alpha=0.3,
)
ax1.fill_between(
    locations,
    means[:, 1] - 3 * stds[:, 1],
    means[:, 1] + 3 * stds[:, 1],
    color="C1",
    alpha=0.3,
)
ax1.fill_between(
    locations,
    means[:, 2] - 3 * stds[:, 2],
    means[:, 2] + 3 * stds[:, 2],
    color="C2",
    alpha=0.3,
)
ax1.plot(sol.locations, sol.states.mean, "o")
fig.suptitle(f"Samples From an EK0 Solution")

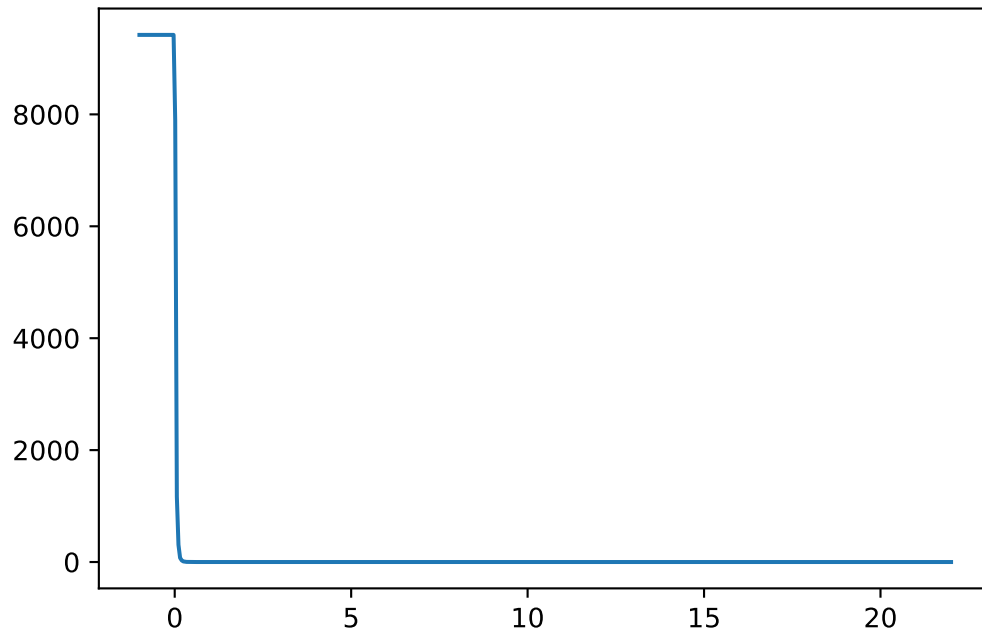
plt.show()

```

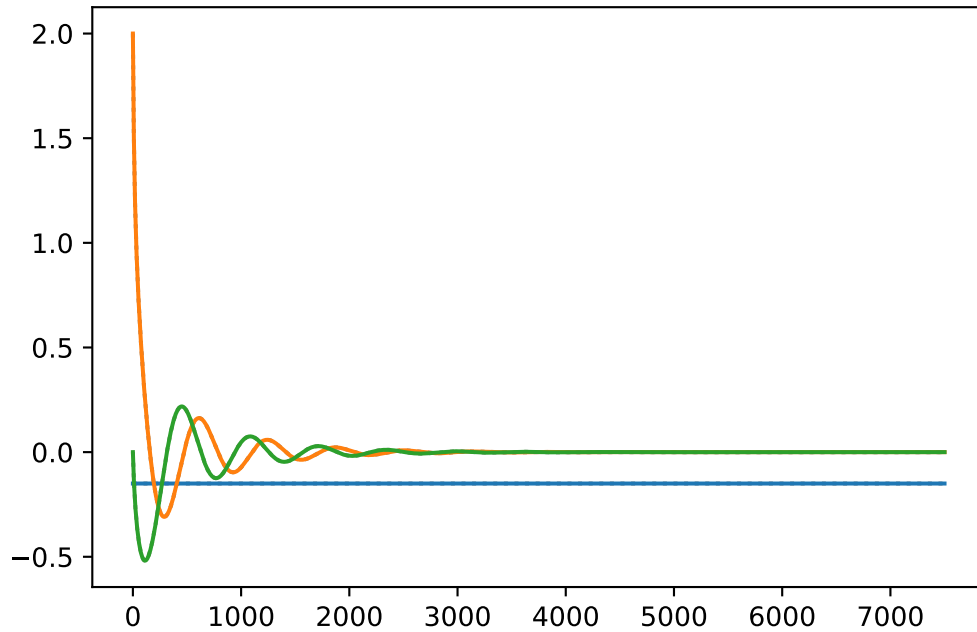
Samples From an EK0 Solution



```
[38]: x = np.linspace(-1, 22, 500)
plt.plot(x, sol.kalman_posterior.diffusion_model(x))
plt.show()
```



```
[39]: plt.plot(samples.mean(axis=0), color="gray", linestyle="dotted")
plt.plot(means)
plt.show()
```

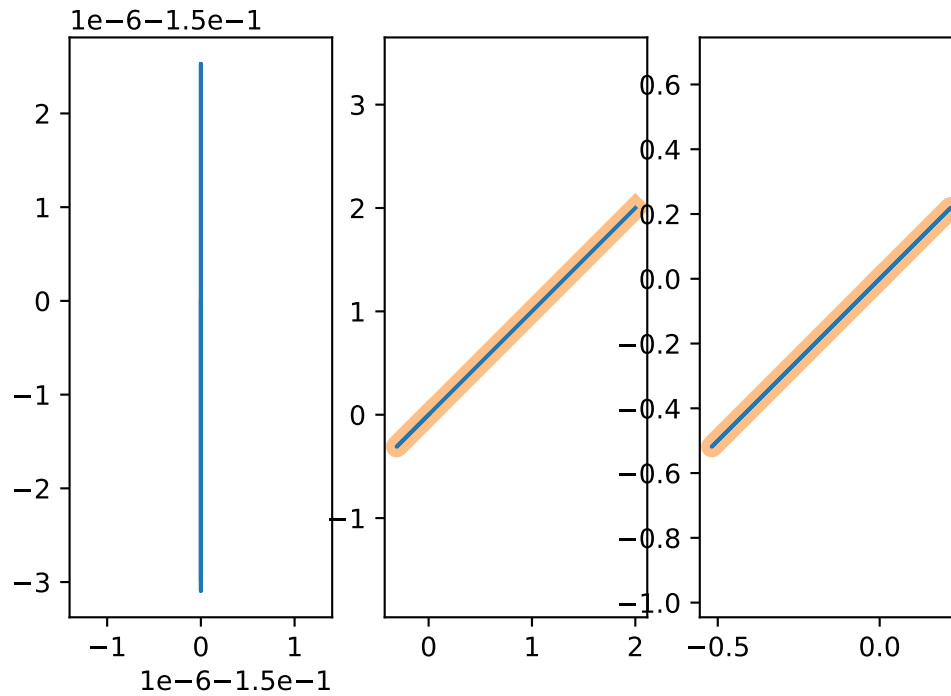


```
[40]: sample_mean = samples.mean(axis=0)
sample_std = samples.std(axis=0)

fig, (ax1, ax2, ax3) = plt.subplots(ncols=3)

ax1.plot(means[:, 0], sample_mean[:, 0])
ax2.plot(means[:, 1], sample_mean[:, 1])
ax3.plot(means[:, 2], sample_mean[:, 2])
ax1.plot(means[:, 0], means[:, 0], alpha=0.5, linewidth=8, zorder=0)
ax2.plot(means[:, 1], means[:, 1], alpha=0.5, linewidth=8, zorder=0)
ax3.plot(means[:, 2], means[:, 2], alpha=0.5, linewidth=8, zorder=0)

ax1.axis("equal")
ax2.axis("equal")
ax3.axis("equal")
plt.show()
```

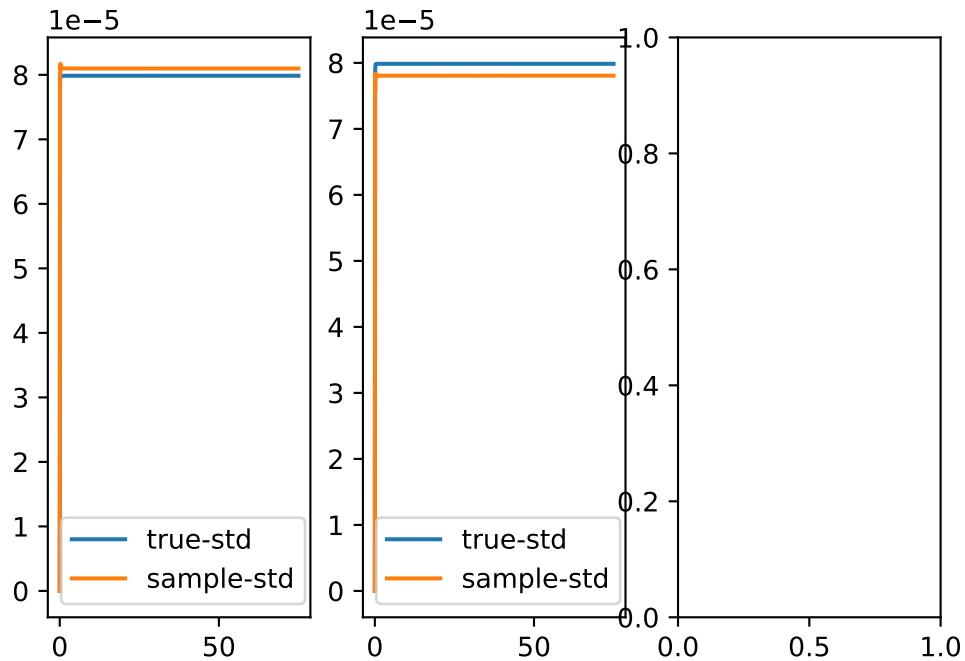


```
[41]: fig, (ax1, ax2, ax3) = plt.subplots(ncols=3)

ax1.plot(locations, stds[:, 0], label="true-std")
ax1.plot(locations, sample_std[:, 0], label="sample-std")
ax1.legend()

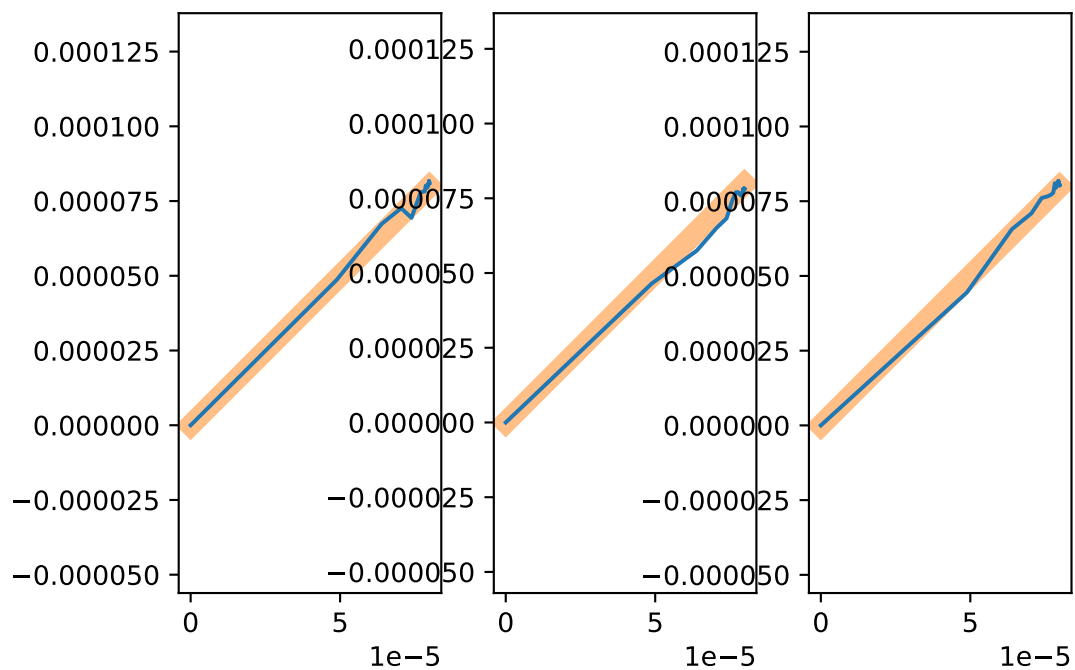
ax2.plot(locations, stds[:, 1], label="true-std")
ax2.plot(locations, sample_std[:, 1], label="sample-std")
ax2.legend()
plt.show()

ax3.plot(locations, stds[:, 2], label="true-std")
ax3.plot(locations, sample_std[:, 2], label="sample-std")
ax3.legend()
plt.show()
```



```
[42]: fig, (ax1, ax2, ax3) = plt.subplots(ncols=3)

ax1.plot(stds[:, 0], sample_std[:, 0])
ax2.plot(stds[:, 1], sample_std[:, 1])
ax3.plot(stds[:, 2], sample_std[:, 2])
ax1.plot(stds[:, 0], stds[:, 0], alpha=0.5, linewidth=8, zorder=0)
ax2.plot(stds[:, 1], stds[:, 1], alpha=0.5, linewidth=8, zorder=0)
ax3.plot(stds[:, 2], stds[:, 2], alpha=0.5, linewidth=8, zorder=0)
ax1.axis("equal")
ax2.axis("equal")
ax3.axis("equal")
plt.show()
```



[]: